Dylan Montgomery
Dr. En Cheng
Database Management 475/575
23 October 2026

Stage 4: Initial Presentation and Demo Report

Team: Lesotho
Captain: Dylan Montgomery (dcm103@uakron.edu)
Member: Andrew Styles (as978@uakron.edu)
Demo: Wednesday, October 28, 2026
Source code: https://github.com/dylcmonty/CPSC475

1. Project Title

Almanac: A Taxonomy-Linked Farm Record Book with Historical Market Pricing

2. Project Summary

Almanac is a web application where a grower keeps a record book of what they planted, harvested, sold, or lost. Every entry is tied to a node in a 4,118-node plant taxonomy. The same taxonomy indexes 47,516 real produce auction prices from four Ohio market report sources, covering 2019 to 2026. Because of that shared key, Almanac can tell a grower what a crop has sold for in a given part of the year, how steady that price has been, and which crops in a plant family have been the best choices for a given planting window. A normal farm record book can't answer those questions.

3. Databases and Software Platforms

Database

The database has twelve tables, fifteen foreign keys, and two views. The full schema with every constraint is in our Stage 3 report. In short:

Group	Tables	Purpose
Reference data	`observer`, `region`, `taxon`, `product`, `product_alias`, `market_lot`, `market_lot_quarantine`, `event_kind`	Data we collected and load ourselves. Users only read it.

Group	Tables	Purpose
Grower data	`farm`, `plot`, `crop_catalog`, `farm_event`	Data that growers create, change, and delete through the site.
Views	`v_priced_lot`, `v_resolution_coverage`	Auction lots joined to their product and taxon with prices in dollars, and the share of lots that have been matched to a product.

`region` and `taxon` reference themselves to form hierarchies. `taxon` stores both a parent pointer, used by recursive queries, and a materialized path such as `1.4.37.210`, used for fast "everything under this node" lookups. Data quality rules are enforced by the database with CHECK constraints, not by application code. For example, an auction lot's average price must lie between its low and high price, and an event with a quantity must also have a unit.

We use PostgreSQL 16 for the live database because our advanced functions need recursive common table expressions and window functions. We use SQLite 3 for local development and tests. The same DDL runs on both except for the auto-increment declaration and the day-of-year function.

Software Platform

Layer	Choice
Language	Python 3.12
Web framework	Flask 3 with Jinja2 server-rendered templates
Database driver	psycopg 3 (PostgreSQL), sqlite3 (development)
SQL	Written by hand, no ORM. All user input is passed as bound parameters.
Charts	Chart.js
Data loaders	Python with pdfplumber (PDF reports) and requests/BeautifulSoup (HTML reports)
Source control	Git and GitHub
Deployment (planned)	Gunicorn behind Nginx on a Linux VPS with HTTPS

4. Where and How We Got Real Data

All reference data is real, collected by our team, and loaded into the database.

Table	Rows	Source
`market_lot`	47,516	Published sale reports from County Line Produce Auction, Mt. Hope Auction, USDA AMS Market News (Mount Hope), and

Table	Rows	Source
		Geauga Growers Produce Auction. 2019-05-03 to 2026-07-22, 560 trading days.
`market_lot_quarantine`	53	Lots from the same reports that failed a CHECK constraint, kept with the reason.
`observer`	4	The four sources above, with domain and reporting granularity (per lot or per product per day).
`taxon`	4,118	A 12-level plant taxonomy compiled from USDA PLANTS and GBIF.
`product`	217	Canonical products we built by grouping the different ways auctions describe the same item.
`product_alias`	one per resolved label	Maps each normalized auction description to a product.
`farm`	238	Real Ohio farm operations from public listings, 131 of them with live web domains.
`region`	2,818	Administrative geography, including all 88 Ohio counties under one Ohio node.

How the market data was loaded. The loader downloads each published report (PDF or web page) and pulls out every lot line: date, description, unit, quantity, and low, high, and average price. Prices are converted to whole cents so no rounding error builds up. Each description is normalized (lowercased, punctuation and extra spaces removed) into `label_norm`, which is the key used to match it to a product. Each row is then inserted. If a row breaks a CHECK constraint, it goes into `market_lot_quarantine` along with the reason, and the constraint stays as written. The 53 quarantined rows are real errors in the source reports. For example, one pumpkin lot was listed with a low of $0.10, a high of $0.30, and an average of $3.18.

Coverage. 44,263 of the 47,516 lots (93.2%) match a product. The other 3,253 have descriptions no product claims yet, such as "10″ Hanging Basket." The `v_resolution_coverage` view reports this number overall and for each source. Advanced Function 3 is designed to raise it.

Grower data. Plots, crop catalogs, and event log entries come from real users. Once the site is public (planned for the week of November 9), we will email Dr. Cheng and invite classmates and local growers to enter their own records. This will happen at least one week before the final presentation.

5. Functions Developed

The basic functions below are built as a Flask application and are in our repository. Each one is covered by an automated test that runs against PostgreSQL. We will demonstrate them live on October 28.

Every query below is parameterized. Values written as `%(name)s` are bound by psycopg and are never pasted into the SQL string. Every write against grower data is limited to the current farm with `farm_id = %(farm_id)s`, so one farm cannot change another farm's records.

5.1 Insert, Update, and Delete

The three grower tables each support full create, edit, and delete.

Farm events (`farm_event`) are the main write surface. Each event records a date, an event kind (plant, harvest, sale, loss), a taxon, an optional plot, an optional product, and an optional quantity with a unit.

```
-- Insert
INSERT INTO farm_event
    (farm_id, plot_id, taxon_id, event_kind_code, product_id,
     event_date, quantity, unit, notes)
VALUES
    (%(farm_id)s, %(plot_id)s, %(taxon_id)s, %(kind)s, %(product_id)s,
     %(event_date)s, %(quantity)s, %(unit)s, %(notes)s)
RETURNING event_id;

-- Update
UPDATE farm_event
SET plot_id = %(plot_id)s, taxon_id = %(taxon_id)s,
    event_kind_code = %(kind)s, product_id = %(product_id)s,
    event_date = %(event_date)s, quantity = %(quantity)s,
    unit = %(unit)s, notes = %(notes)s
WHERE event_id = %(event_id)s AND farm_id = %(farm_id)s;

-- Delete
DELETE FROM farm_event
WHERE event_id = %(event_id)s AND farm_id = %(farm_id)s;
```

Plots (`plot`) are named pieces of ground on a farm, with an optional area.

```
INSERT INTO plot (farm_id, name, area_sq_m)
VALUES (%(farm_id)s, %(name)s, %(area_sq_m)s)
RETURNING plot_id;

UPDATE plot SET name = %(name)s, area_sq_m = %(area_sq_m)s
WHERE plot_id = %(plot_id)s AND farm_id = %(farm_id)s;

DELETE FROM plot
WHERE plot_id = %(plot_id)s AND farm_id = %(farm_id)s;
```

Crop catalog (`crop_catalog`) lists the taxa a farm grows. Its primary key is `(farm_id, taxon_id)`, so adding a crop that is already listed does nothing.

```
INSERT INTO crop_catalog (farm_id, taxon_id, notes)
VALUES (%(farm_id)s, %(taxon_id)s, %(notes)s)
ON CONFLICT (farm_id, taxon_id) DO NOTHING;

UPDATE crop_catalog SET notes = %(notes)s
WHERE farm_id = %(farm_id)s AND taxon_id = %(taxon_id)s;

DELETE FROM crop_catalog
WHERE farm_id = %(farm_id)s AND taxon_id = %(taxon_id)s;
```

What the database enforces. These rules come from the schema and apply to every write, whatever page it comes from:

- Deleting a plot keeps its events. Their `plot_id` becomes NULL (`ON DELETE SET NULL`), so harvest history is not lost when a field is renamed or retired.
- A taxon with recorded events cannot be deleted (`ON DELETE RESTRICT`).
- Two plots on the same farm cannot share a name (`UNIQUE (farm_id, name)`).
- A quantity must be zero or more, and an event with a quantity must have a unit.
- A plot's area must be greater than zero.

When a write breaks one of these rules, the application catches the database's integrity error and shows the user which rule failed. It does not show a server error.

5.2 Search and Retrieval

Combined catalog search. One search box checks scientific names, common names, and product titles together. Results are sorted by how much price evidence each match has, so a search for a crop lists the cultivated plant that has auction history before a wild relative that has none.

```
SELECT t.taxon_id, t.rank, t.scientific_name, t.common_name,
       COUNT(v.lot_id) AS lots
FROM taxon t
LEFT JOIN v_priced_lot v ON v.taxon_id = t.taxon_id
WHERE t.scientific_name ILIKE %(pattern)s
   OR t.common_name     ILIKE %(pattern)s
   OR EXISTS (SELECT 1 FROM product p
              WHERE p.taxon_id = t.taxon_id
                AND p.title ILIKE %(pattern)s)
GROUP BY t.taxon_id, t.rank, t.scientific_name, t.common_name, t.depth
ORDER BY lots DESC, t.depth, t.scientific_name
LIMIT 50;
```

`%(pattern)s` is the user's text wrapped in `%` wildcards by the application.

Seasonal price curve. For one crop, this returns the lowest, average, and highest price in each calendar month across all seven seasons. The page draws it as a Chart.js line chart. Counting distinct years shows the user which months are backed by many seasons and which by only one.

```
SELECT EXTRACT(MONTH FROM v.sale_date)::INT          AS month,
       COUNT(*)                                      AS lots,
       COUNT(DISTINCT EXTRACT(YEAR FROM v.sale_date)) AS seasons,
       MIN(v.low_price)                              AS floor_price,
       ROUND(AVG(v.avg_price), 2)                    AS mean_price,
       MAX(v.high_price)                             AS ceiling_price
FROM v_priced_lot v
WHERE v.taxon_id = %(taxon_id)s
GROUP BY EXTRACT(MONTH FROM v.sale_date)
ORDER BY month;
```

Farm activity history. This shows one farm's events in a date range. The event kind and crop filters are optional, and leaving either one blank turns it off.

```
SELECT e.event_id, e.event_date, k.label AS event_kind,
       t.common_name, t.scientific_name,
       pl.name AS plot, p.title AS product,
       e.quantity, e.unit, e.notes
FROM farm_event e
JOIN event_kind k     ON k.event_kind_code = e.event_kind_code
JOIN taxon t          ON t.taxon_id = e.taxon_id
LEFT JOIN plot pl     ON pl.plot_id = e.plot_id
LEFT JOIN product p   ON p.product_id = e.product_id
WHERE e.farm_id = %(farm_id)s
  AND e.event_date BETWEEN %(date_from)s AND %(date_to)s
  AND (CAST(%(kind)s AS VARCHAR) IS NULL OR e.event_kind_code = %(kind)s)
  AND (CAST(%(taxon_id)s AS INTEGER) IS NULL OR e.taxon_id = %(taxon_id)s)
ORDER BY e.event_date DESC, e.event_id DESC;
```

Harvest compared with market price (multi-table join). This is our main join. For each harvest a farm has recorded, it finds what that crop sold for at auction within 30 days on either side of the harvest date. It joins the event log, farm, event kind, taxonomy, product lexicon, and market lot tables, plus `product_alias`, which links each auction lot to its product.

```
SELECT f.name                          AS farm,
       e.event_date                    AS harvested_on,
       COALESCE(t.common_name, t.scientific_name) AS crop,
       e.quantity, e.unit,
       p.title                         AS market_product,
       COUNT(l.lot_id)                 AS lots_in_window,
       ROUND(AVG(l.avg_cents) / 100.0, 2) AS mean_price,
       MIN(l.low_cents)  / 100.0       AS low_price,
       MAX(l.high_cents) / 100.0       AS high_price
FROM farm_event e
JOIN farm f           ON f.farm_id = e.farm_id
JOIN event_kind k     ON k.event_kind_code = e.event_kind_code
                     AND k.is_yield
JOIN taxon t          ON t.taxon_id = e.taxon_id
JOIN product p        ON p.taxon_id = e.taxon_id
JOIN product_alias a  ON a.product_id = p.product_id
JOIN market_lot l     ON l.label_norm = a.label_norm
                     AND l.sale_date BETWEEN e.event_date - 30
                                         AND e.event_date + 30
WHERE e.farm_id = %(farm_id)s
GROUP BY f.name, e.event_id, e.event_date, t.common_name, t.scientific_name,
         e.quantity, e.unit, p.title
ORDER BY e.event_date DESC, lots_in_window DESC;
```

Only events whose kind has `is_yield = TRUE` (harvests) are included. Matching on `taxon_id` instead of crop names means a harvest recorded as *Allium cepa* finds every auction lot for any onion product, however the auction worded it.

6. Planned Advanced Functions

We plan three advanced functions. Two meet the 475 requirement, and the third meets the 575 requirement. All three rely on the taxonomy being a real hierarchy linked to the price data.

Advanced Function 1: Recursive Taxonomic Price Roll-up (week of October 26)

Problem. A user picks any node in the taxonomy, such as a genus, family, or order, and wants prices for everything under it. The tree is twelve levels deep, and the number of levels below a node isn't known in advance, so a fixed number of joins can't express the query. Running one query per level from the application would need many round trips and would move the work out of the database.

Approach. A recursive common table expression starts at the chosen node and repeatedly joins children on `parent_taxon_id` until it reaches the bottom of the subtree. The result is joined through the product lexicon to the price data and aggregated, all in one statement:

```
WITH RECURSIVE subtree AS (
    SELECT taxon_id, scientific_name, common_name, depth
    FROM taxon
    WHERE taxon_id = %(root_taxon_id)s
    UNION ALL
    SELECT c.taxon_id, c.scientific_name, c.common_name, c.depth
    FROM taxon c
    JOIN subtree s ON c.parent_taxon_id = s.taxon_id
)
SELECT s.taxon_id, s.scientific_name, s.common_name,
       COUNT(v.lot_id)            AS lots,
       ROUND(AVG(v.avg_price), 2) AS mean_price,
       MIN(v.low_price)           AS floor_price,
       MAX(v.high_price)          AS ceiling_price
FROM subtree s
JOIN v_priced_lot v ON v.taxon_id = s.taxon_id
GROUP BY s.taxon_id, s.scientific_name, s.common_name
ORDER BY lots DESC;
```

Result so far. Run on the order Asparagales, the query walks 198 descendant nodes, finds the 6 that have priced products, and returns 2,679 lots. Onion alone accounts for 1,382 lots at a $9.63 mean. "Onion," "garlic," and "shallot" share no text, so matching on names alone would not group them.

Remaining work. A page that lets the user pick a node from the tree, a table of results with a total row, and a chart of the subtree's prices over time. We will also compare the recursive query with a lookup on the materialized path (`path LIKE '1.4.37.%'`) and report which is faster.

Advanced Function 2: Crop Decision Engine (week of November 2)

Problem. "What should I plant in the first week of May?" A useful answer has to compare a calendar window across several years, weigh price against how much it swings, leave out crops with too little data to judge, and stick to crops the grower can actually grow.

Approach. A chain of common table expressions:

1. Get the taxonomy subtree the grower chooses, using the same recursive CTE as Advanced Function 1.
2. Select every priced lot whose day of year falls inside the requested window, in any year. The window may wrap past December 31.
3. For each product, compute the number of lots, the number of distinct years, the mean price, and the standard deviation.

4. Drop any product with fewer than 12 lots or fewer than 2 distinct years.
5. Compute a stability score that lowers a crop's mean price by its volatility. We currently plan to use mean ÷ (1 + coefficient of variation).
6. Rank the products with `RANK() OVER (ORDER BY score DESC)`.

Every candidate must resolve to a taxon. Our first version ranked "Watering can" first, because these auctions also sell merchandise. Requiring a taxon removes merchandise without a hand-written exclusion list.

Result so far. For late April with the root set at flowering plants, the query returns 25 ranked candidates, led by strawberry, rose, and marigold. That matches what sells at Ohio auctions in spring. With the root set at Asparagales, it returns two: garlic and asparagus.

Remaining work. A form with a date window, a taxonomy node, and minimum evidence settings, and a results table where each row links to that crop's seasonal price curve.

Advanced Function 3: Taxon Resolution Ladder for Free-Text Market Labels (week of November 2)

Problem. 3,253 auction lots have descriptions that don't match any product yet, so their prices don't appear in any query above. Simple substring matching gives wrong answers. "Wild garlic" is not garlic, and a description that matches two products equally well should not be assigned to whichever one the database returns first.

Approach. Each unresolved label is tried against four rungs, from most to least confident:

1. An exact match between the normalized label and a product slug.
2. A token match against product titles.
3. A token match against taxonomic common names.
4. A token match against scientific binomials.

At each rung, exactly one candidate means a match is proposed. Zero candidates sends the label to the next rung. More than one candidate stops the search and marks the label **ambiguous**, so it never falls through to a looser rung that would give a confident wrong answer. "Not found" and "ambiguous" are reported separately because they are fixed in different ways.

A person approves every proposed match on a review screen before it is written. The approval writes one row to `product_alias`, recording the rung that proposed it in `match_rung`. This instantly links every past and future lot with that label to the product. The ladder reads from the taxonomy and product tables and has no hard-coded rules for particular crops, so it keeps working as new sources and labels arrive.

How we will measure it. We will report the `v_resolution_coverage` percentage before and after, overall and for each source, along with how many labels ended as ambiguous and how many as not found.

7. Next Steps

Week	Work
October 19	Submit this report (October 23).
October 26	Stage 4 demo (October 28). Advanced Function 1 page. Source comparison page (one crop in one period, side by side across the four sources).
November 2	Advanced Function 2 page. Advanced Function 3 ladder and review screen.
November 9	Stage 5 progress report. Deploy publicly, email Dr. Cheng, and start inviting users.
November 16	Collect real user data and fix bugs from feedback.
November 23	Feature freeze, full testing, presentation practice.
November 30	Stage 6 final demo with the basic functions and all three advanced functions.
December 7	Stage 7 final submission.